



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Kazuyuki Ishii
2025-11-24



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
- Summary of all results

Introduction

- Project background and context
- Problems you want to find answers

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - the API to extract information using identification numbers in the launch data.
 - <https://api.spacexdata.com/v4/rockets/>
 - <https://api.spacexdata.com/v4/launchpads/>
 - <https://api.spacexdata.com/v4/payloads/>
 - <https://api.spacexdata.com/v4/cores/>
 - <https://api.spacexdata.com/v4/launches/past>
- Perform data wrangling
 - Collected data was enhanced by generating a landing outcome label derived from the analysis and summary of outcome-related features.

Methodology

Executive Summary

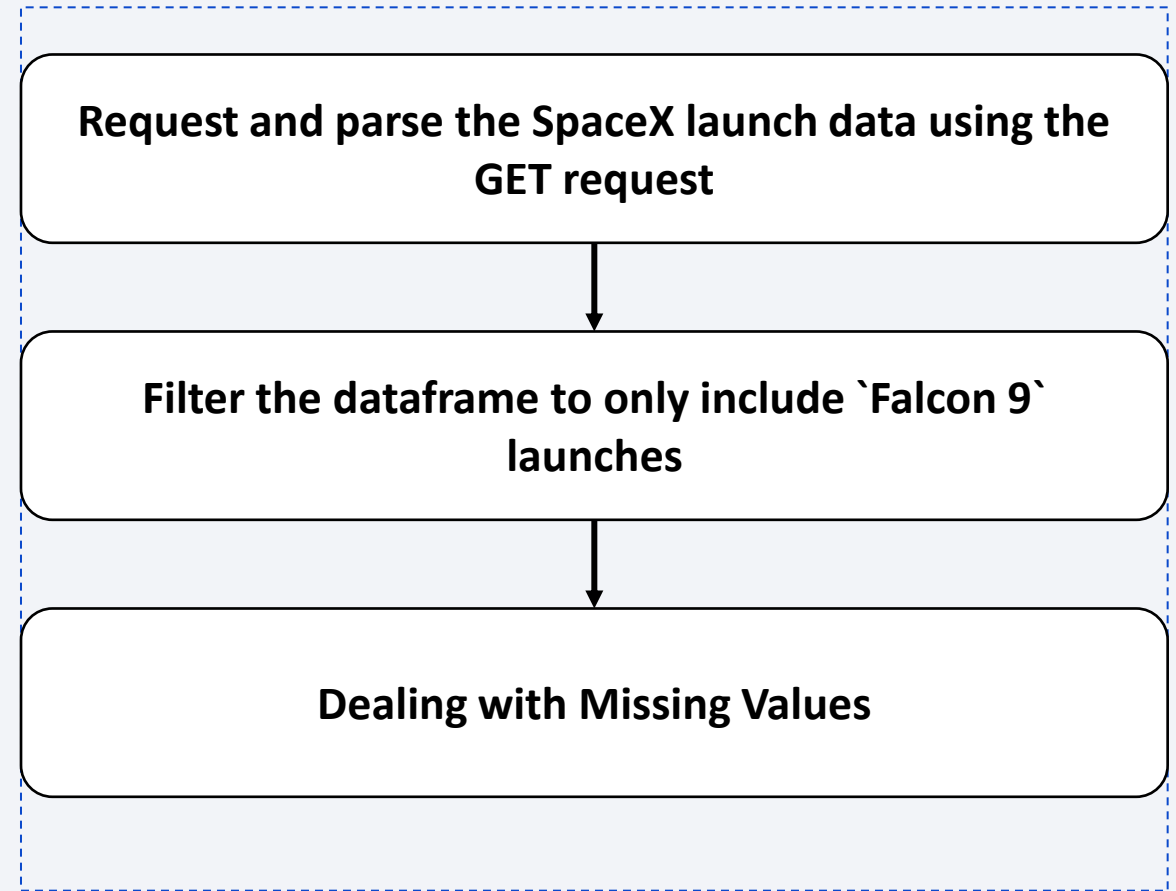
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - The data collected up to this stage were normalized, split into training and test datasets, and assessed using four distinct classification models, with each model's accuracy evaluated under various parameter combinations.

Data Collection

- Describe how data sets were collected.
 - Datasets were collected from SpaceX API:
 - <https://api.spacexdata.com/v4/rockets/>
 - <https://api.spacexdata.com/v4/launchpads/>
 - <https://api.spacexdata.com/v4/payloads/>
 - <https://api.spacexdata.com/v4/cores/>
 - <https://api.spacexdata.com/v4/launches/past>
 - from Wikipedia for web scraping to collect Falcon 9 historical launch records:
 - https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches

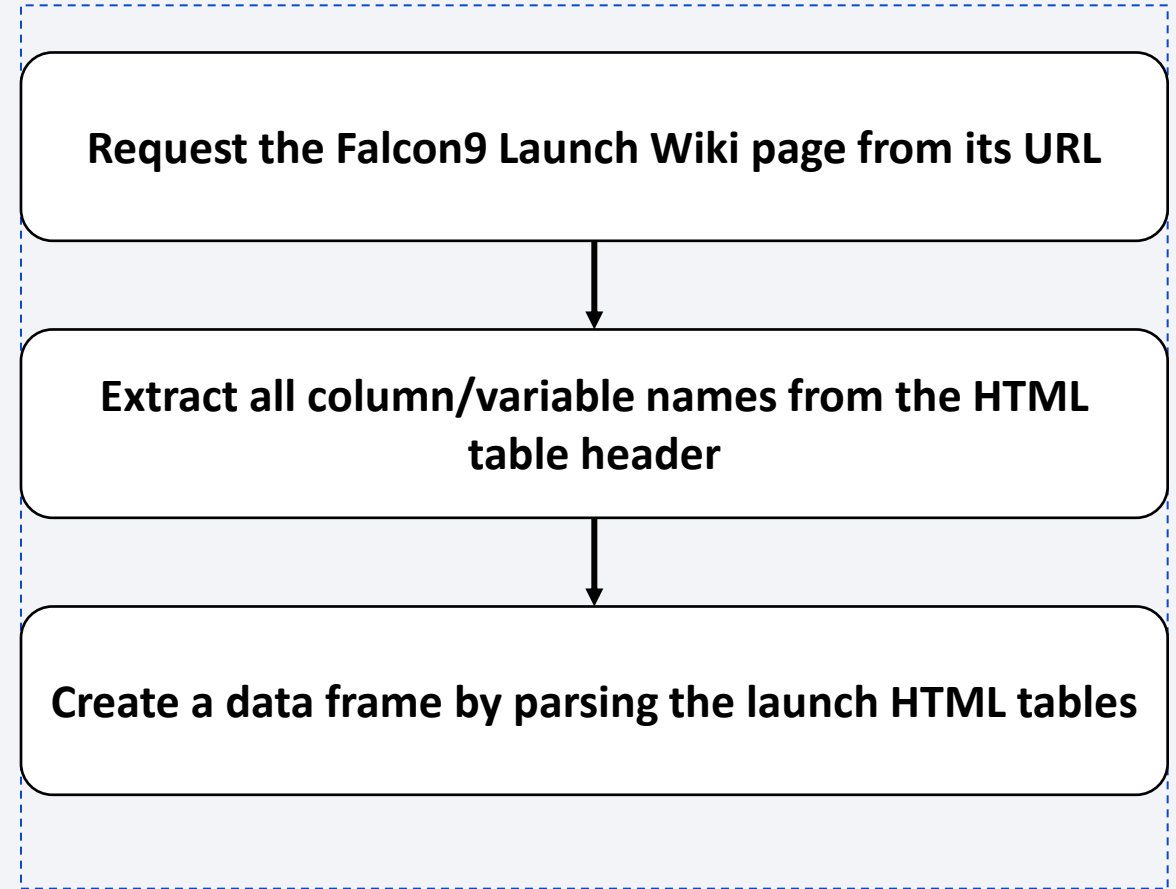
Data Collection – SpaceX API

- Present your data collection with SpaceX REST calls using key phrases and flowcharts
- Add the GitHub URL of the completed SpaceX API calls notebook (**must include completed code cell and outcome cell**), as an external reference and peer-review purpose
 - [Applied-Data-Science-Capstone/Module 1/jupyter-labs-spacex-data-collection-api.ipynb](https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/jupyter-labs-spacex-data-collection-api.ipynb) at main · kishii239/Applied-Data-Science-Capstone



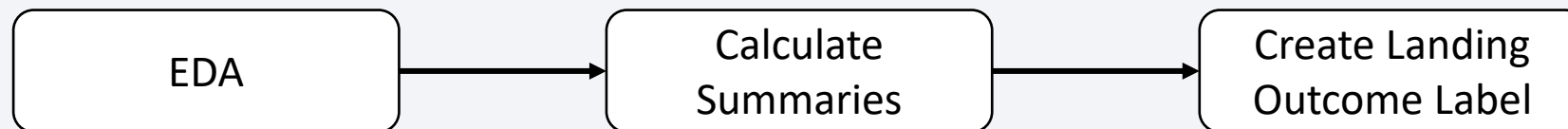
Data Collection - Scraping

- Present your web scraping process using key phrases and flowcharts
- Add the GitHub URL of the completed web scraping notebook, as an external reference and peer-review purpose
 - https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module_1/jupyter-labs-webscraping.ipynb



Data Wrangling

- Describe how data were processed
- You need to present your data wrangling process using key phrases and flowcharts
 1. Some Exploratory Data Analysis (EDA) was performed on the dataset.
 2. The summary launches per site, occurrences of each orbit and occurrences of mission outcome per orbit type were calculated.
 3. The landing outcome label was created from Outcome column.



- Add the GitHub URL of your completed data wrangling related notebooks, as an external reference and peer-review purpose
 - [https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module 1/labs-jupyter-spacex-Data%20wrangling.ipynb](https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module%201/labs-jupyter-spacex-Data%20wrangling.ipynb)

EDA with Data Visualization

- Summarize what charts were plotted and why you used those charts
 - Names of the unique launch sites in the space mission: The notebook identifies the unique values in the LaunchSite column.
 - Top 5 launch sites whose name begins with the string 'CCA': The notebook's data cleaning/feature engineering does not specifically perform this query, but the unique launch sites identified are: CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, and CCAFS SLC-40. Only two begin with 'CCA', so listing the "Top 5" is contextually inaccurate.
 - Total payload mass carried by boosters launched by NASA (CRS): This is a specific query on the PayloadMass column filtered by missions where the Customer is NASA (CRS), which is a standard analysis.
 - Average payload mass carried by booster version F9 v1.1: This is a calculation of the mean of PayloadMass for rows where BoosterVersion is F9 v1.1.
 - Date when the first successful landing outcome in ground pad was achieved: This requires sorting the data by date and filtering for the first instance of 'Success (ground pad)' in the LandingOutcome column.
 - Names of the boosters which have success in drone ship and have payload mass between 4000 and 6000 kg: This is a filter operation on BoosterVersion based on LandingOutcome ('Success (drone ship)') and a range filter on PayloadMass.
 - Total number of successful and failure mission outcomes: This is typically a count (or value counts) of the Class column (where 1 is success and 0 is failure).
 - Names of the booster versions which have carried the maximum payload mass: This involves finding the maximum value in PayloadMass and identifying the corresponding BoosterVersion names.
 - Failed landing outcomes in droneship, their booster versions, and launch site names for in year 2015: This involves filtering by LandingOutcome ('Failure (drone ship)'), the mission Date (in 2015), and then extracting BoosterVersion and LaunchSite.
 - Rank of the count of landing outcomes (such as Failure (droneship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20: This involves filtering by the date range, performing a value_counts() on the LandingOutcome column, and then ranking the results.
- Add the GitHub URL of your completed EDA with data visualization notebook, as an external reference and peer-review purpose
 - https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module_2/edadataviz.ipynb

EDA with SQL

- Using bullet point format, summarize the SQL queries you performed
 - To explore the data, scatterplots and bar charts were employed to visualize relationships between various feature pairs, including Payload Mass vs. Flight Number, Launch Site vs. Flight Number, Launch Site vs. Payload Mass, Orbit vs. Flight Number, and Payload Mass vs. Orbit.
- Add the GitHub URL of your completed EDA with SQL notebook, as an external reference and peer-review purpose
 - [https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module 2/edadataviz.ipynb](https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module%202/edadataviz.ipynb)

Build an Interactive Map with Folium

- Summarize what map objects such as markers, circles, lines, etc. you created and added to a folium map
- Explain why you added those objects
 - Folium Maps were used with markers, circles, lines, and marker clusters to visualize spatial data. Markers represent specific points such as launch sites. Circles highlight areas around particular coordinates, such as the NASA Johnson Space Center. Marker clusters group related events within the same coordinate, such as multiple launches from a single site. Lines illustrate distances between two coordinates.
- Add the GitHub URL of your completed interactive map with Folium map, as an external reference and peer-review purpose
 - [https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module 3/lab_jupyter_launch_site_location.ipynb](https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module%203/lab_jupyter_launch_site_location.ipynb)

Build a Dashboard with Plotly Dash

- Summarize what plots/graphs and interactions you have added to a dashboard
- Explain why you added those plots and interactions
 - We developed an interactive dashboard using Plotly Dash to visualize launch data. The dashboard includes pie charts displaying the total number of launches by site and scatter plots illustrating the relationship between launch outcomes and payload mass (kg) across different booster versions.
- Add the GitHub URL of your completed Plotly Dash lab, as an external reference and peer-review purpose
 - https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module_3/spacex-dash-app.py

Predictive Analysis (Classification)

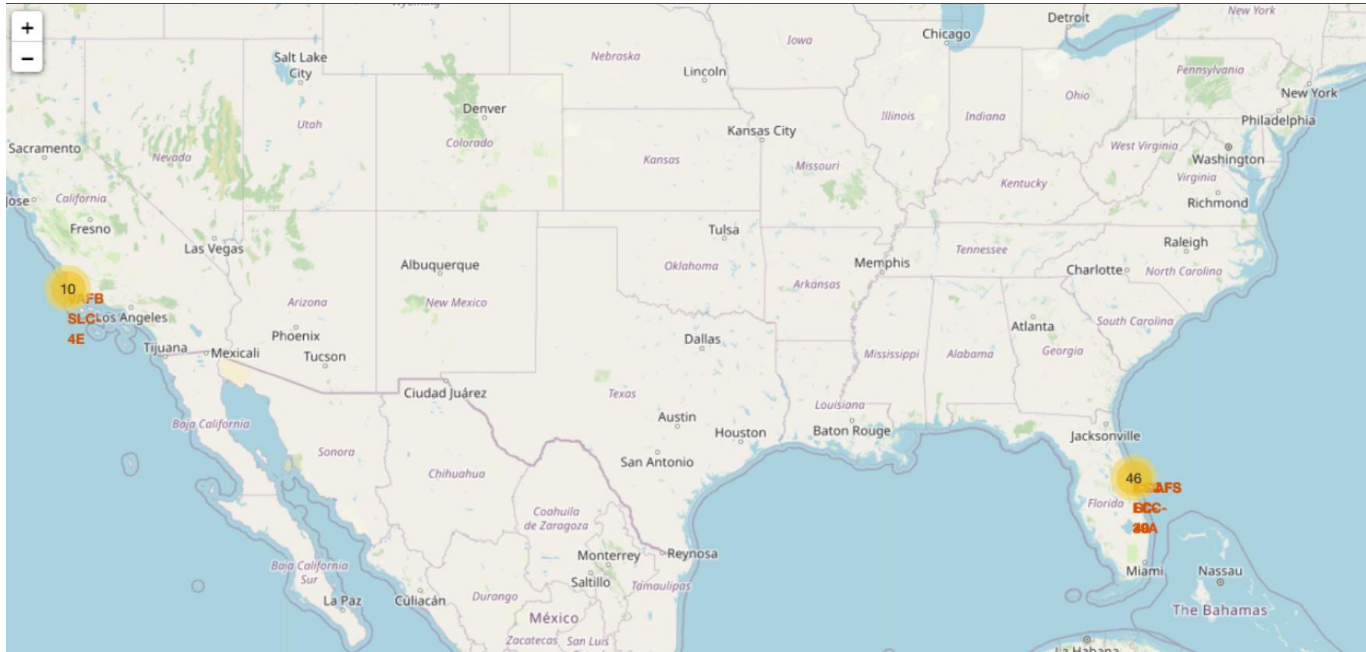
- Summarize how you built, evaluated, improved, and found the best performing classification model
- You need present your model development process using key phrases and flowchart
- Add the GitHub URL of your completed predictive analysis lab, as an external reference and peer-review purpose
 - [https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module 4/SpaceX Machine%20Learning%20Prediction Part 5.ipynb](https://github.com/kishii239/Applied-Data-Science-Capstone/blob/main/Module%204/SpaceX%20Machine%20Learning%20Prediction%20Part%205.ipynb)

Results

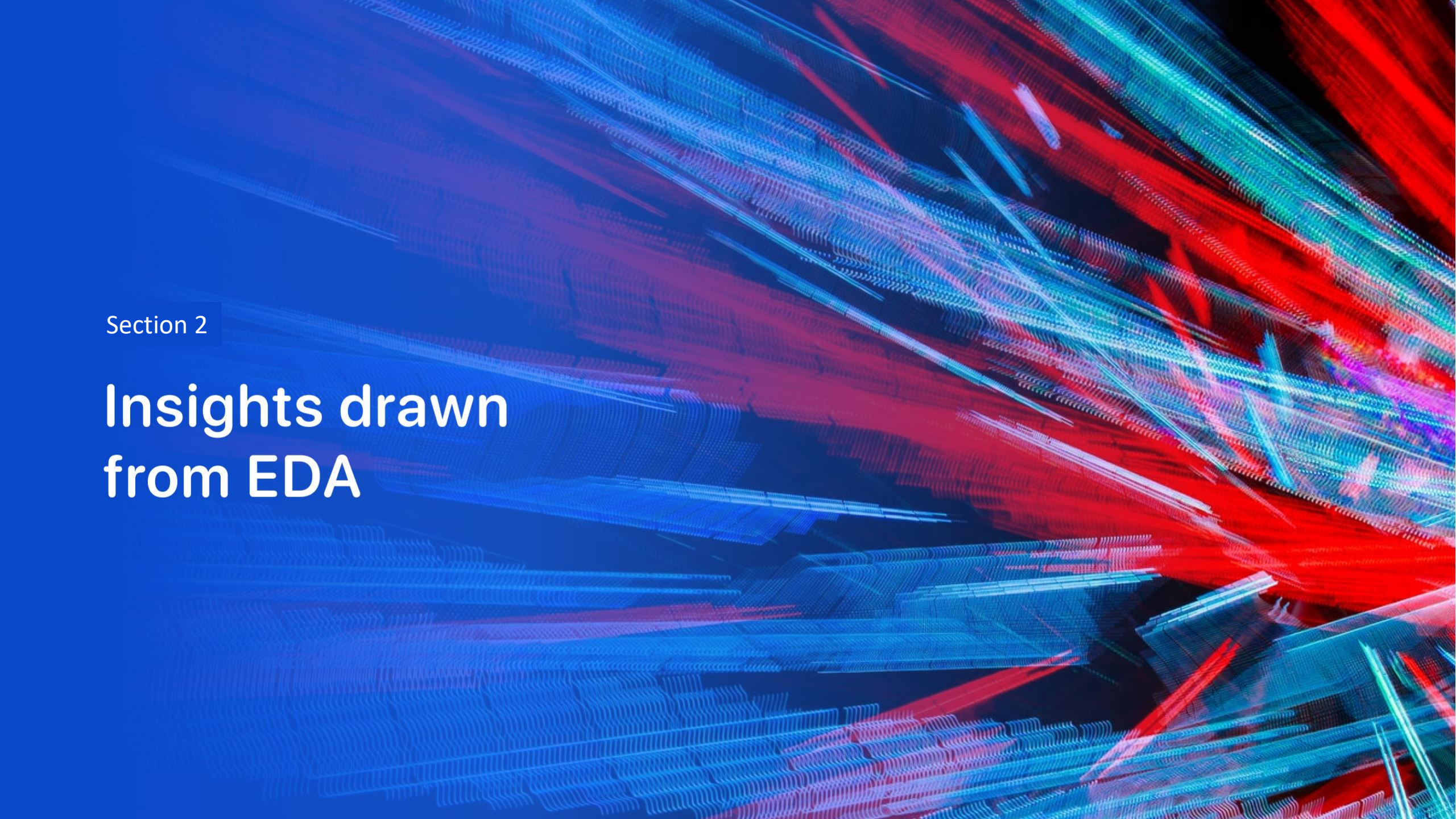
- Exploratory data analysis results
 - SpaceX operates four different launch sites.
 - The first launches were conducted for both SpaceX and NASA.
 - The average payload capacity of the Falcon 9 v1.1 booster is 2,928 kg.
 - The first successful landing occurred in 2015, five years after the maiden launch.
 - Several versions of the Falcon 9 booster successfully landed on drone ships carrying payloads above the average.
 - Almost 100% of the missions have been successful.
 - Two booster versions failed to land on drone ships in 2015: Falcon 9 v1.1 B1012 and Falcon 9 v1.1 B1015.
 - The number of successful landings improved steadily over the years.

Results

- Interactive analytics demo in screenshots



- Predictive analysis results
 - Most launches happened at the launch site in Florida.

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower half of the image. The overall effect is dynamic and technological.

Section 2

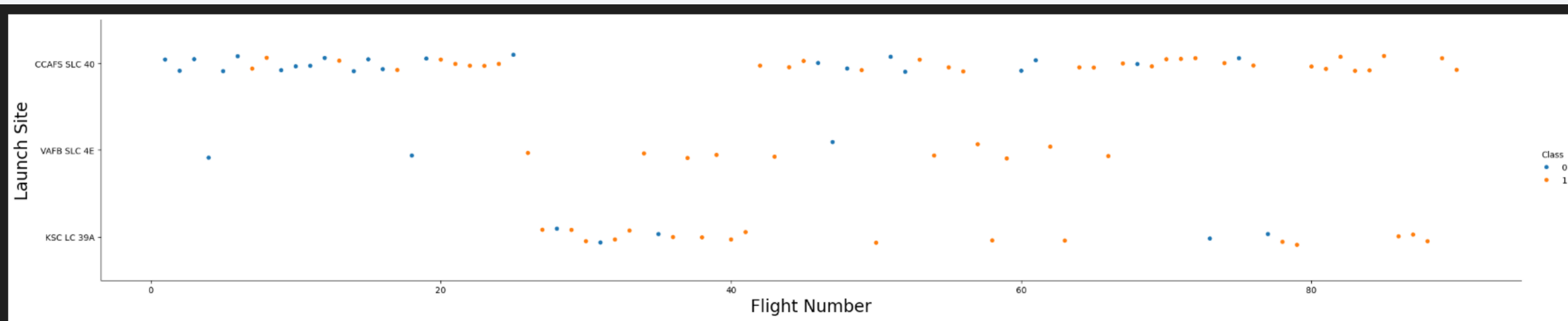
Insights drawn from EDA

Flight Number vs. Launch Site

- Show a scatter plot of Flight Number vs. Launch Site

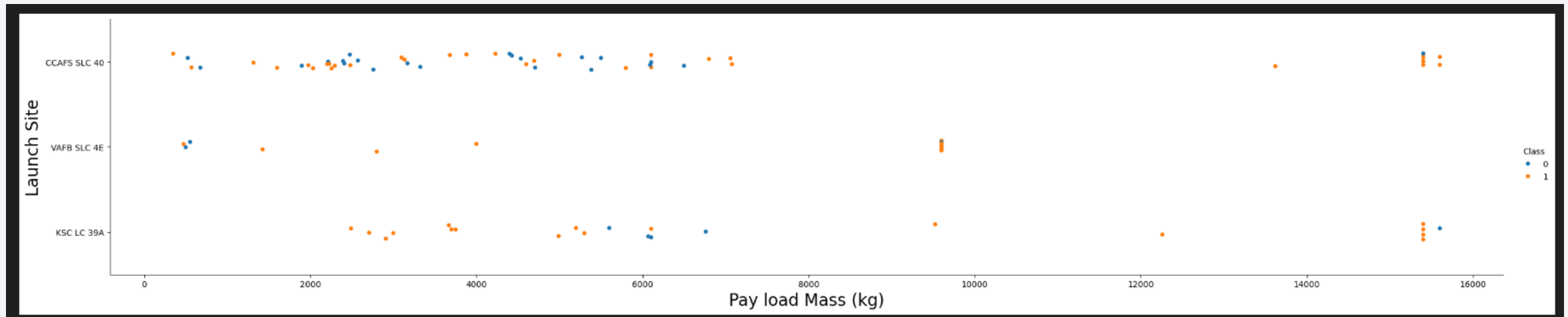
```
sns.catplot(x="FlightNumber",y="LaunchSite",hue='Class',data=df, aspect=5)  
plt.xlabel("Flight Number",fontsize=20)  
plt.ylabel("Launch Site",fontsize=20)  
plt.show()
```

- Show the screenshot of the scatter plot with explanations



Payload vs. Launch Site

- Show a scatter plot of Payload vs. Launch Site
- Show the screenshot of the scatter plot with explanations



- The above is a scatter plot of Launch site vs Payload Mass with success/failure indicated in the legend.
- It can be observed that for higher payload mass, success rates are relatively higher.

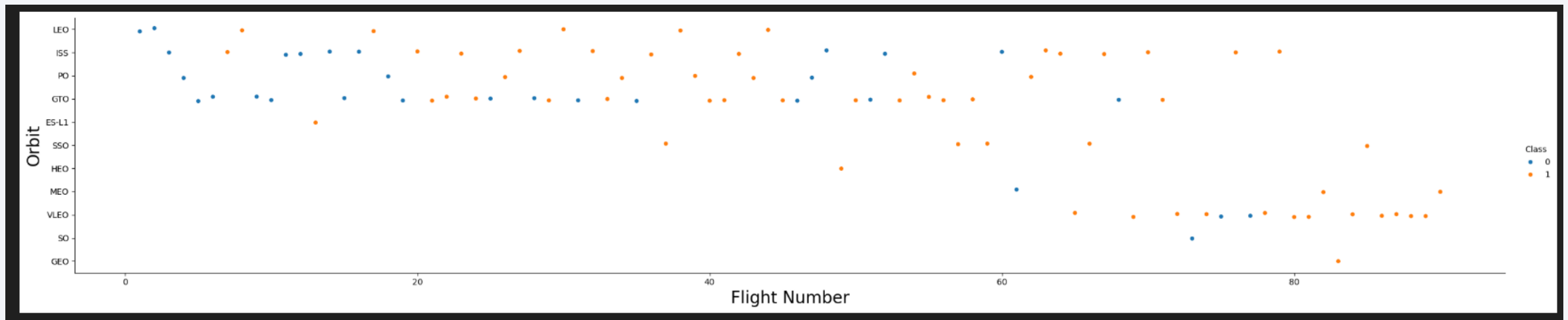
Success Rate vs. Orbit Type

- Show a bar chart for the success rate of each orbit type
- Show the screenshot of the scatter plot with explanations
 - From the plot, we can see that ES-L1, GEO, HEO, SSO, VLEO had the most success rate.



Flight Number vs. Orbit Type

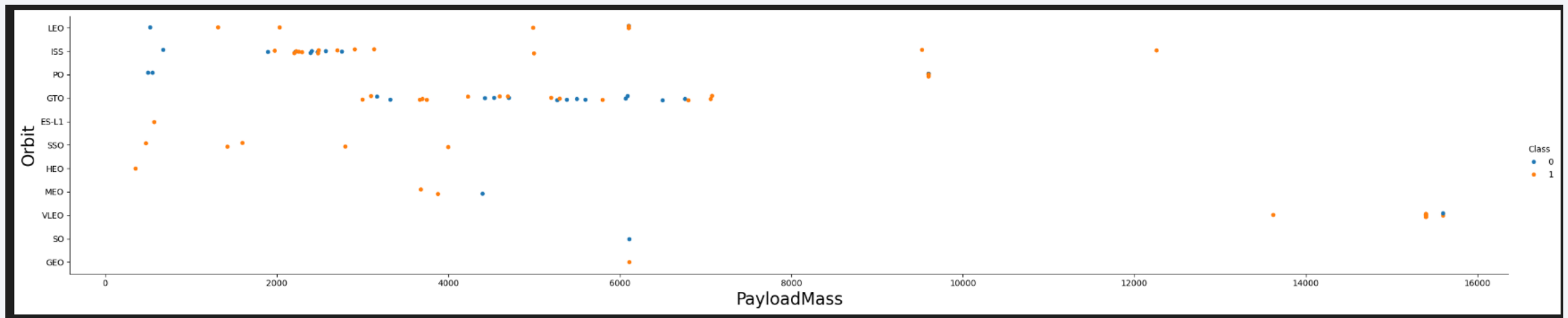
- Show a scatter point of Flight number vs. Orbit type
- Show the screenshot of the scatter plot with explanations



- We observe that in the LEO orbit, success is related to the number of flights whereas in the GTO orbit, there is no relationship between flight number and the orbit.

Payload vs. Orbit Type

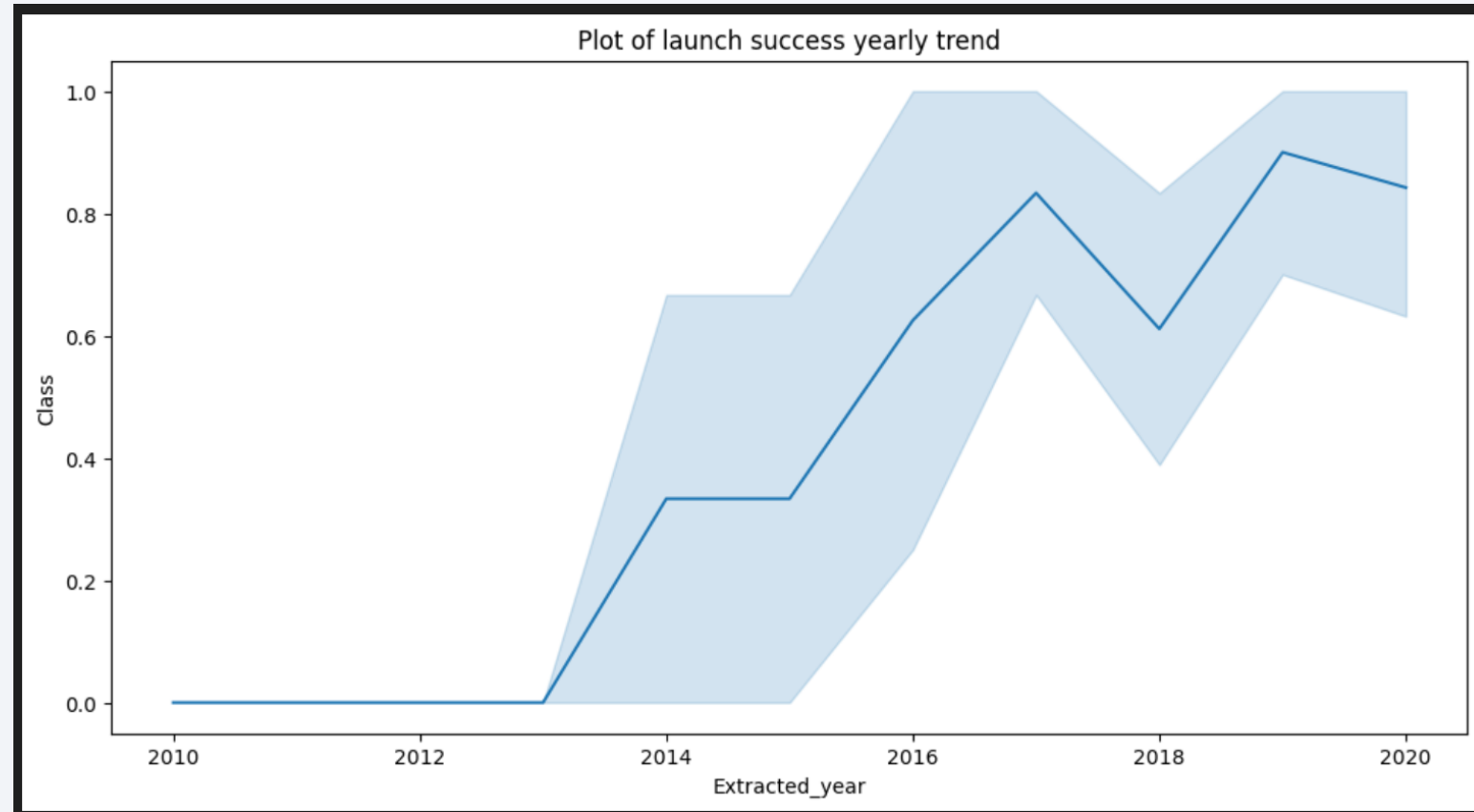
- Show a scatter point of payload vs. orbit type
- Show the screenshot of the scatter plot with explanations



- We can observe that with heavy payloads, the successful landing are more for PO, LEO and ISS orbits.

Launch Success Yearly Trend

- Show a line chart of yearly average success rate
- Show the screenshot of the scatter plot with explanations
 - we can observe that success rate since 2013 kept on increasing till 2020.



All Launch Site Names

- Find the names of the unique launch sites
- Present your query result with a short explanation here
 - We used the key word **DISTINCT** to show only unique launch sites from the SpaceX data.

```
%sql select distinct Launch_Site from SPACEXTABLE;
```

```
* sqlite:///my\_data1.db
```

```
Done.
```

```
Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with `CCA`
- Present your query result with a short explanation here
 - We used the query above to display 5 records where launch sites begin with `CCA`

```
%sql select * from SPACEXTABLE where Launch_Site like 'CCA%' limit 5;
```

Python

```
* sqlite:///my\_data1.db
```

Done.

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

- Calculate the total payload carried by boosters from NASA
- Present your query result with a short explanation here
 - We calculated the total payload carried by boosters from NASA as 45596 using the query below

```
[12] %sql select sum(PAYLOAD_MASS_KG_) from SPACE_TABLE where Customer = 'NASA (CRS)';  
... * sqlite:///my_data1.db  
Done.  
... sum(PAYLOAD_MASS_KG_)  
45596
```

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1
- Present your query result with a short explanation here
 - We calculated the average payload mass carried by booster version F9 v1.1 as 2928.4

```
[13] %sql select avg(PAYLOAD_MASS_KG_) from SPACEXTABLE where Booster_Version = 'F9 v1.1';  
... * sqlite:///my\_data1.db  
Done.  
... avg(PAYLOAD_MASS_KG_)  
2928.4
```

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad
- Present your query result with a short explanation here
 - We observed that the dates of the first successful landing outcome on ground pad was 22nd December 2015

```
%sql select min(Date) from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my\_data1.db
```

```
Done.
```

```
min(Date)
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000
- Present your query result with a short explanation here
 - We used the **WHERE** clause to filter for boosters which have successfully landed on drone ship and applied the **AND** condition to determine successful landing with payload mass greater than 4000 but less than 6000

```
%sql select Booster_Version from SPACEXTABLE\  
where Landing_Outcome = "Success (drone ship)" and PAYLOAD_MASS__KG_ between 4000 and 6000;
```

Python

```
* sqlite:///my\_data1.db
```

Done.

Booster_Version

F9 FT B1022

F9 FT B1026

F9 FT B1021.2

F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes
- Present your query result with a short explanation here
 - We used wildcard to filter for **GROUP BY** Mission_Outcome was a success or a failure.

```
%sql SELECT Mission_Outcome, COUNT(*) as count from SPACEXTABLE\  
GROUP BY Mission_Outcome ORDER BY count DESC;
```

Python

```
* sqlite:///my\_data1.db
```

Done.

Mission_Outcome	count
Success	98
Success (payload status unclear)	1
Success	1
Failure (in flight)	1

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass
- Present your query result with a short explanation here
 - We determined the booster that have carried the maximum payload using a subquery in the WHERE clause and the MAX() function.

```
%sql select Booster_Version from SPACEXTABLE\
where PAYLOAD_MASS_KG_ = (select max(PAYLOAD_MASS_KG_) from SPACEXTABLE)
```

Python

```
* sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- Present your query result with a short explanation here
 - We used a combinations of the **WHERE** clause and **AND** conditions to filter for failed landing outcomes in drone ship, their booster versions, and launch site names for year 2015.

```
%sql select substr(Date, 6, 2) as month, Landing_Outcome, Booster_Version, Launch_Site from SPACEXTABLE\
where substr(Date, 0, 5) = '2015' and Landing_Outcome = 'Failure (drone ship)'
```

Python

```
* sqlite:///my\_data1.db
Done.
```

month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order
- Present your query result with a short explanation here
 - We selected Landing outcomes and the COUNT of landing outcomes from the data and used the WHERE clause to filter for landing outcomes BETWEEN 2010-06-04 to 2010-03-20.
 - We applied the GROUP BY clause to group the landing outcomes and the ORDER BY clause to order the grouped landing outcome in descending order.

```
%sql select Landing_Outcome, count(*) as count_outcomes \
from SPACEXTABLE\
where Date between '2010-06-04' and '2017-03-20'\
group by Landing_Outcome\
order by count_outcomes Desc;
```

* [sqlite:///my_data1.db](#)
Done.

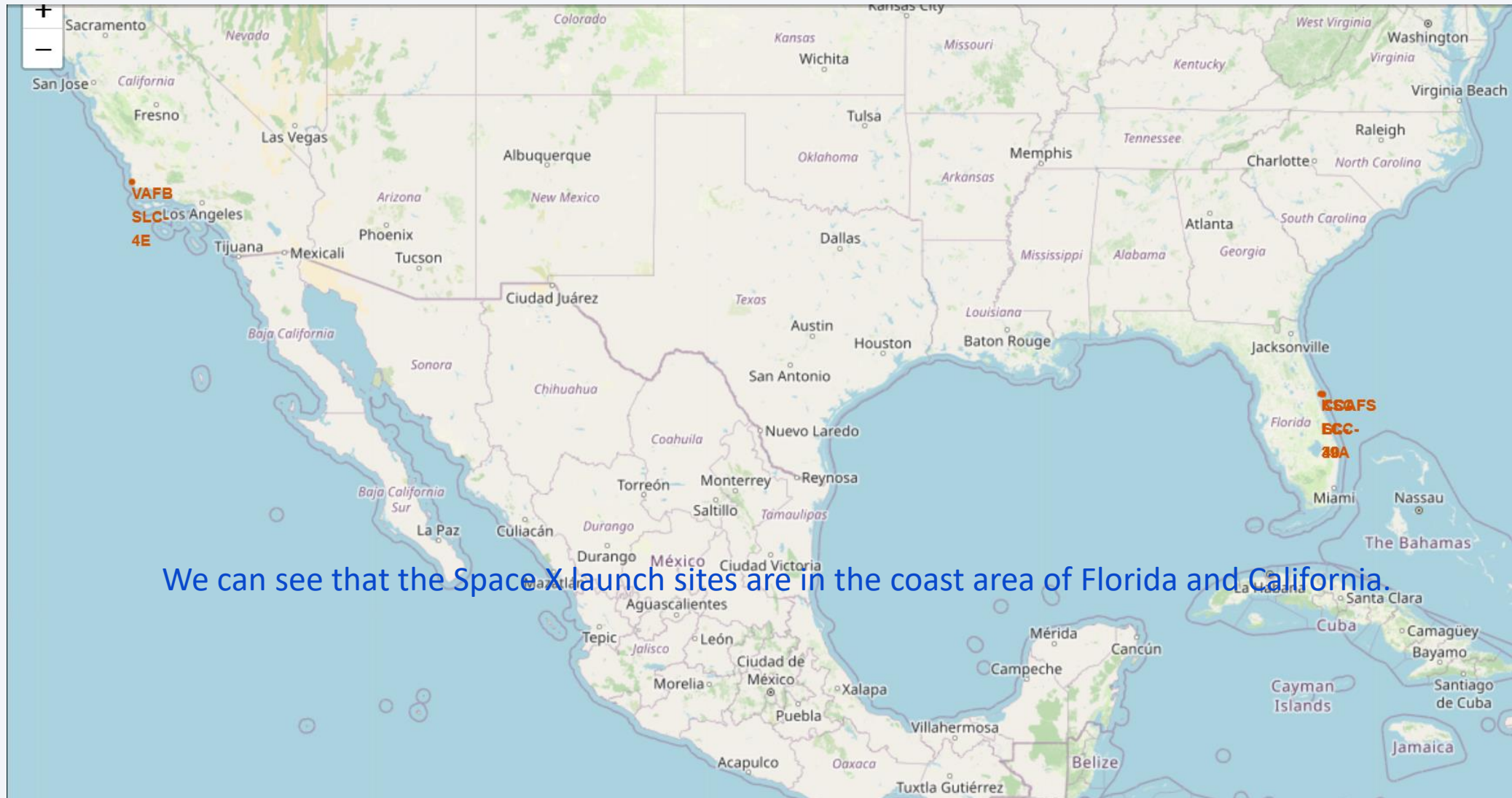
Landing_Outcome	count_outcomes
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

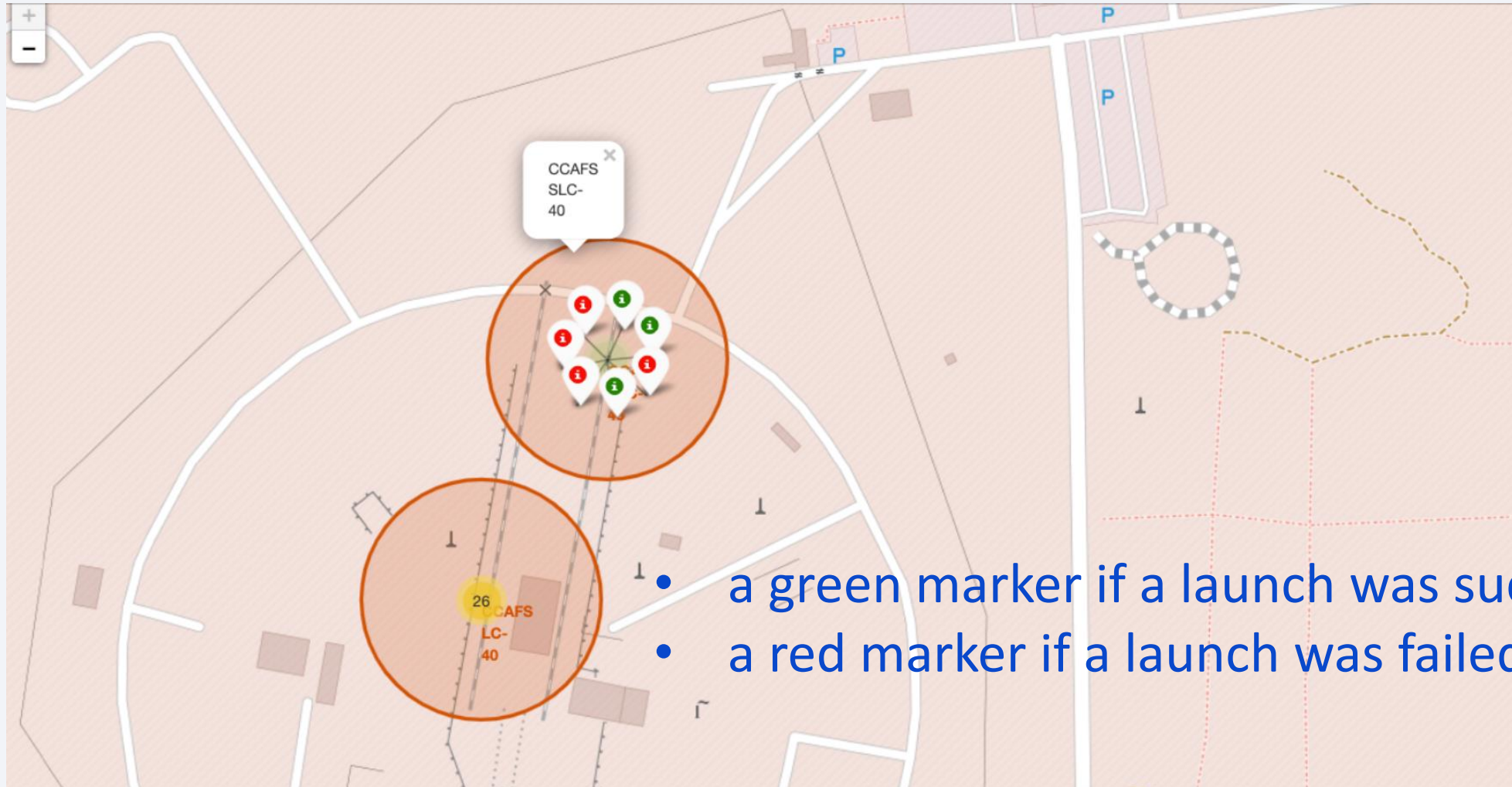
Launch Sites Proximities Analysis

Mark all launch sites on a map

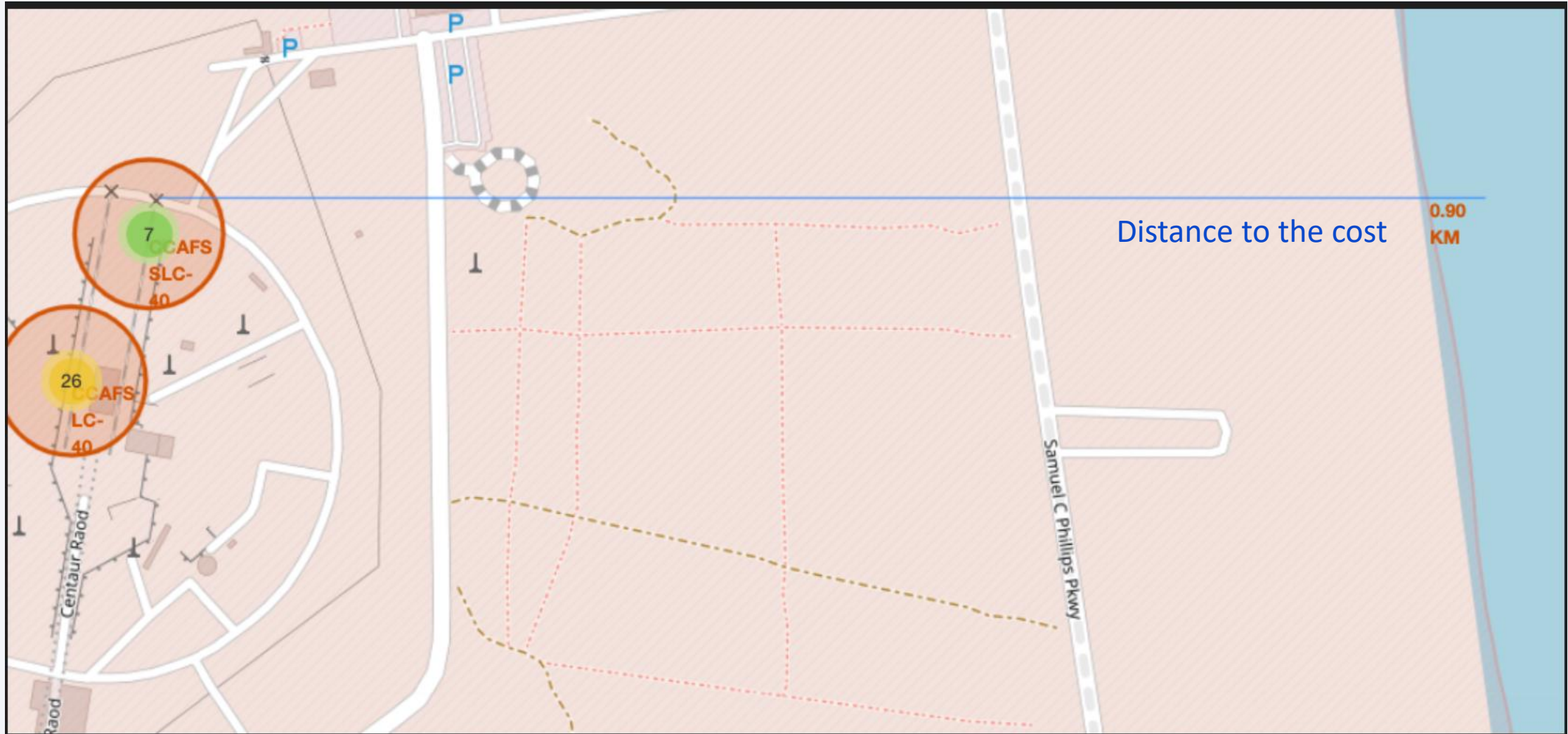


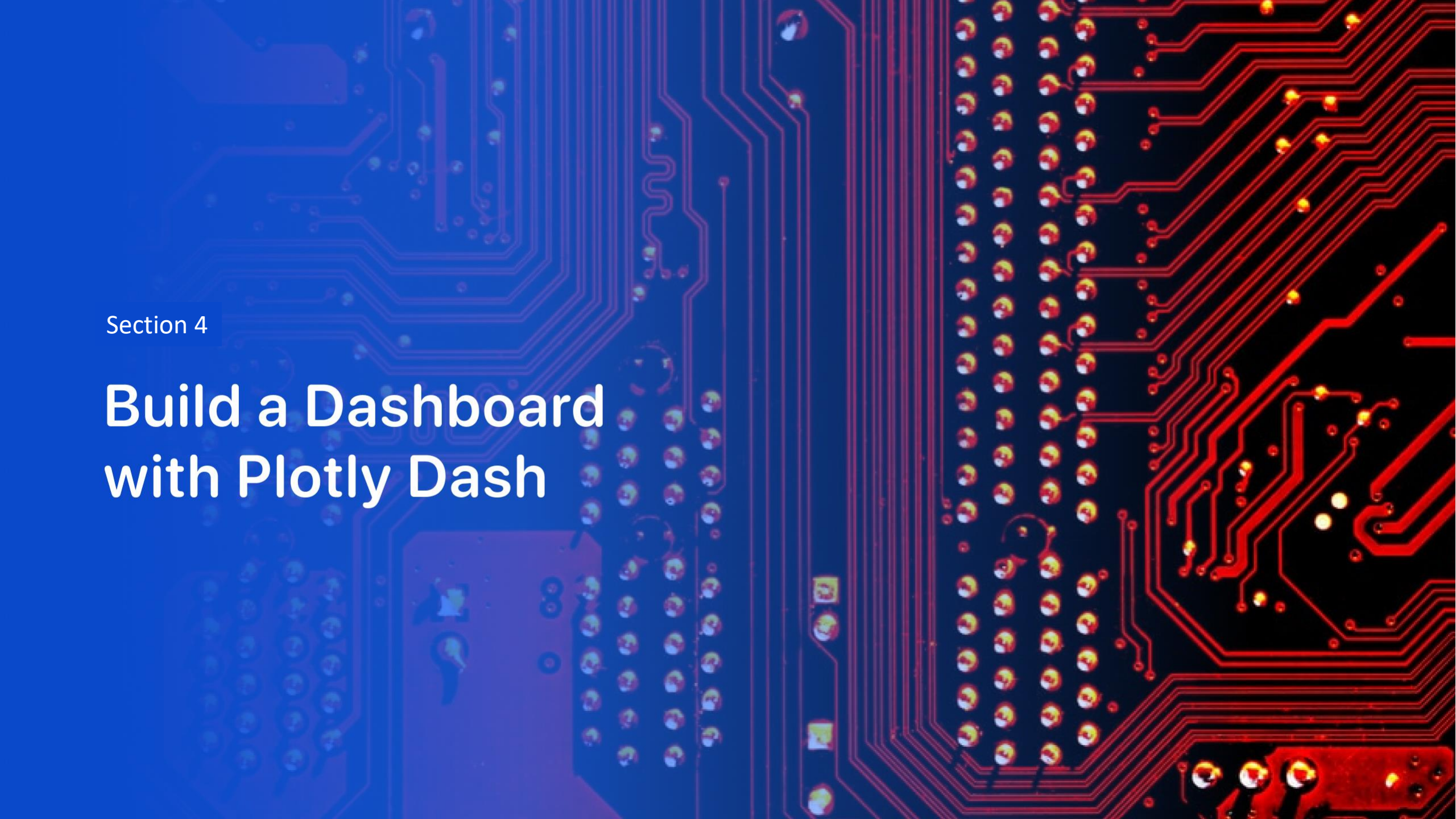
We can see that the Space X launch sites are in the coast area of Florida and California.

Mark the success/failed launches for each site on the map



Calculate the distances between a launch site to its proximities



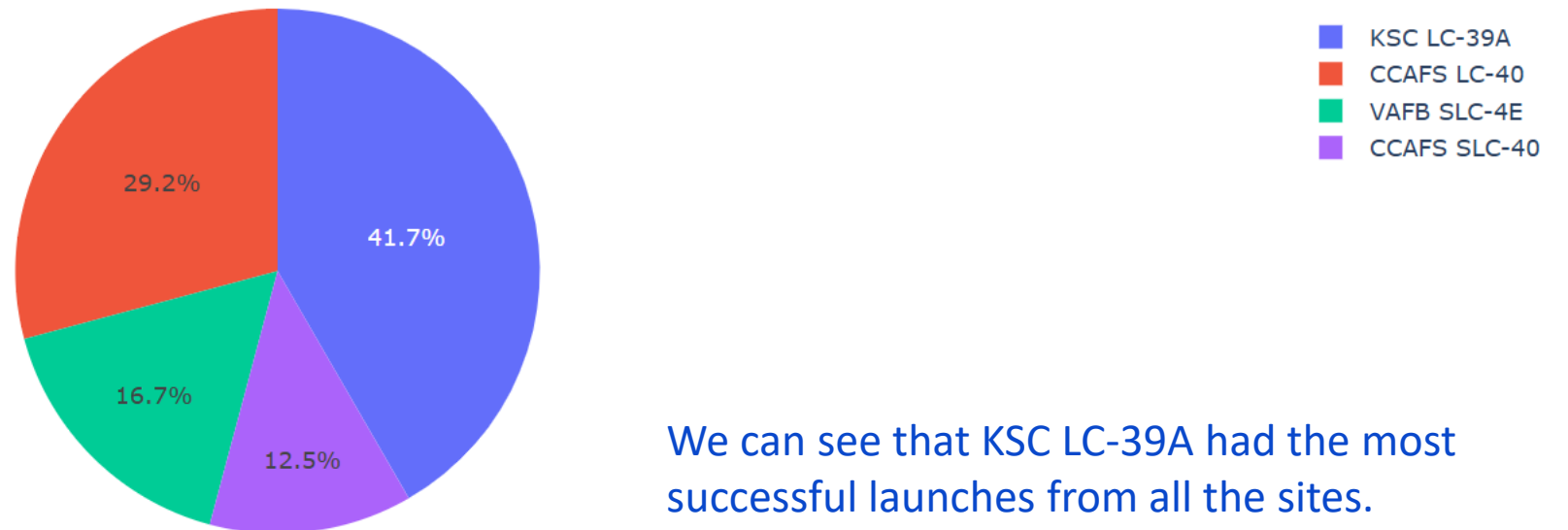


Section 4

Build a Dashboard with Plotly Dash

Pie chart showing the success percentage achieved by each launch site

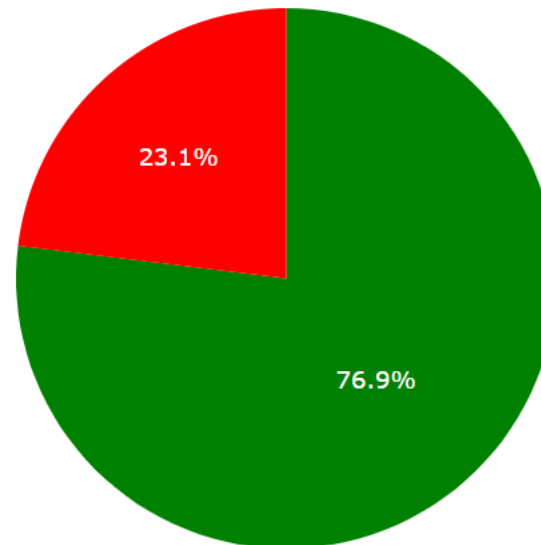
Total Successful Launches by Site



We can see that KSC LC-39A had the most successful launches from all the sites.

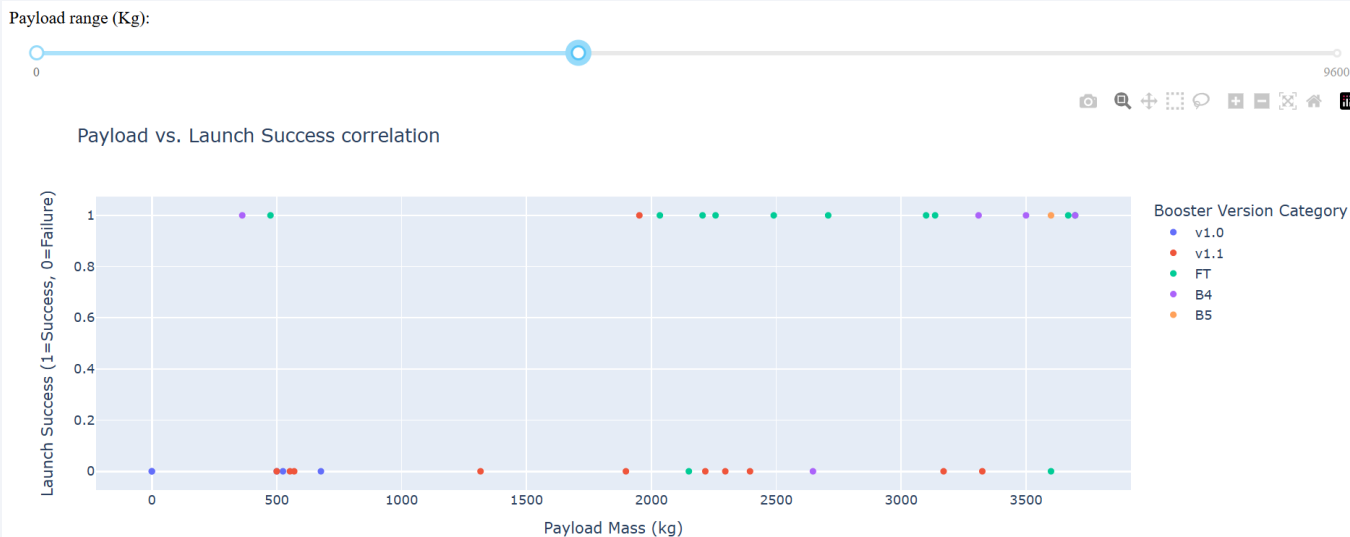
Pie chart showing the Launch site with the highest launch success ratio

Success vs. Failure for site KSC LC-39A

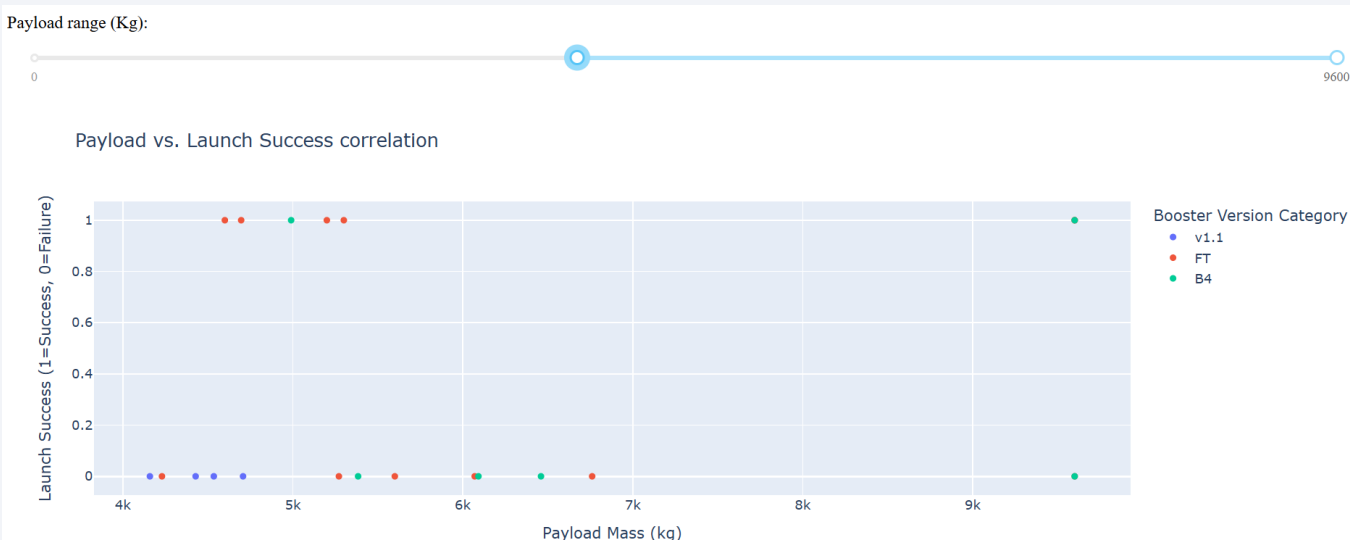


We can see that KSC LC-39A had the most successful launches from all the sites.

Scatter plot of Payload vs Launch Outcome for all sites, with different payload selected in the range slider



- Low Weighted Payload :
0 kg ~ 4000 kg



- Heavy Weighted Payload:
4000 kg ~ 10000 kg

We can see the success rates for low weighted payloads is higher than the heavy weighted payloads.

Section 5

Predictive Analysis (Classification)

Classification Accuracy

```
# Gather the best scores from each model
logreg_score = logreg_cv.best_score_
svm_score = svm_cv.best_score_
tree_score = tree_cv.best_score_
knn_score = knn_cv.best_score_

# Print out each model's best score
print("Logistic Regression Best Accuracy:", logreg_score)
print("Support Vector Machine Best Accuracy:", svm_score)
print("Decision Tree Best Accuracy:", tree_score)
print("K-Nearest Neighbors Best Accuracy:", knn_score)

# Determine and print the best performing model
best_model_name = None
best_score = max(logreg_score, svm_score, tree_score, knn_score)

if best_score == logreg_score:
    best_model_name = "Logistic Regression"
elif best_score == svm_score:
    best_model_name = "Support Vector Machine"
elif best_score == tree_score:
    best_model_name = "Decision Tree"
else:
    best_model_name = "K-Nearest Neighbors"

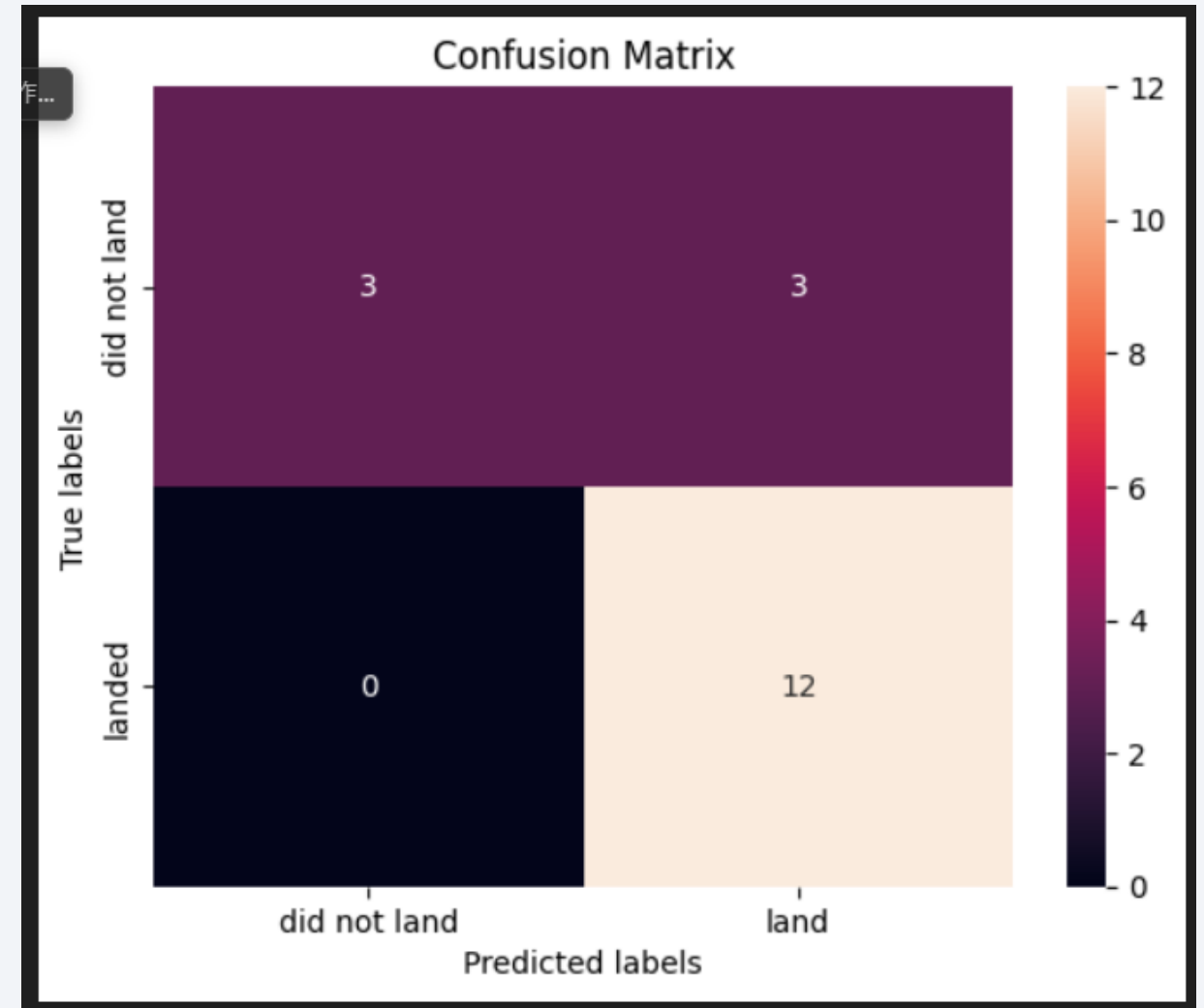
print(f"The best performing model is {best_model_name} with an accuracy of {best_score:.4f}")
```

```
Logistic Regression Best Accuracy: 0.8464285714285713
Support Vector Machine Best Accuracy: 0.8482142857142856
Decision Tree Best Accuracy: 0.8732142857142856
K-Nearest Neighbors Best Accuracy: 0.8482142857142858
The best performing model is Decision Tree with an accuracy of 0.8732
```

The decision tree classifier is the model with the highest classification accuracy

Confusion Matrix

- The confusion matrix for the decision tree classifier shows that the classifier can distinguish between the different classes.
- The major problem is the false positives .i.e., unsuccessful landing marked as successful landing by the classifier.



Conclusions

We can conclude that:

- The higher the number of flights at a launch site, the greater its success rate.
- The launch success rate increased steadily from 2013 to 2020.
- The orbits ES-L1, GEO, HEO, SSO, and VLEO showed the highest success rates.
- KSC LC-39A recorded the most successful launches among all sites.
- The Decision Tree Classifier proved to be the best machine learning algorithm for this task.

Appendix

- Include any relevant assets like Python code snippets, SQL queries, charts, Notebook outputs, or data sets that you may have created during this project

Thank you!

